

Serilog + Elasticsearch + Kibana - Beginner's Guide

Part 1: The Problem (Why We Need Logging)

Imagine you're running a lemonade stand with 6 friends helping you. Each friend has a different job:

- One makes the lemonade
- One takes orders
- One handles money
- One delivers to customers
- One manages coupons
- One handles customer accounts

Problem: When something goes wrong (like a customer says "I paid but didn't get my lemonade"), how do you figure out what happened? You can't watch everyone all the time!

Solution: You ask everyone to write down everything they do in a notebook:

- "At 2:30 PM, I took an order from John"
- "At 2:31 PM, I made lemonade for John"
- "At 2:32 PM, I delivered to John"

Now when there's a problem, you just read the notebooks to see what went wrong!

In software: Your microservices are like your friends. Serilog is the notebook system that helps them write down what they do.

Part 2: What is Serilog? (The Notebook Writer)

Serilog is like a smart notebook system for your computer programs. Here's what it does:

Without Serilog:

```
Console.WriteLine("Order created for John");
```

This just prints to the screen - once the program stops, the message is gone forever!

With Serilog:

```
Log.Information("Order created for {UserName}", "John");  
Log.Warning("Payment failed for order {OrderId}", 123);  
Log.Error("Database connection lost!");
```

Serilog can:

- Save messages to multiple places (console, files, databases)
- Add extra information automatically (time, which service, environment)
- Format messages nicely
- Filter messages (show only important ones in production)

Think of it like: A smart assistant that writes down everything your programs do, and can save those notes in different places depending on what you need.

Part 3: What is Elasticsearch? (The Super-Fast Search Engine)

Problem: When you have thousands of notebooks (logs) from many services, finding a specific problem is like searching for a needle in a haystack!

Elasticsearch is like a super-fast search engine for your logs. Think of it like Google, but for your application logs.

What it does:

- Stores millions of log messages
- Can search through them in milliseconds
- Can find patterns (like "show me all errors in the last hour")
- Can count things (like "how many orders failed today?")

Real-world example:

- Without Elasticsearch: You open 100 text files and search manually (takes hours)
- With Elasticsearch: You type "error" and it shows all errors in 0.1 seconds

Think of it like: A magical library where you can find any book in seconds, even if there are millions of books!

Part 4: What is Kibana? (The Beautiful Dashboard)

Problem: Elasticsearch is powerful but it's just text - like reading raw code. It's hard to see patterns and understand what's happening at a glance.

Kibana is like a beautiful dashboard that shows your logs in charts, graphs, and tables. Think of it like the dashboard in your car, but for your application.

What it shows you:

- **Charts:** "How many errors per hour?" (line graph going up/down)
- **Pie charts:** "Which service has the most errors?" (Catalog 30%, Basket 20%, etc.)
- **Maps:** "Where are users coming from?" (world map with dots)
- **Tables:** "Recent errors" (sorted list with details)
- **Real-time:** Watch logs appear live as they happen

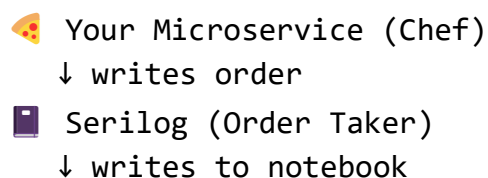
Real-world example:

- Without Kibana: You see raw text like "ERROR at 2:30: Database failed"
- With Kibana: You see a red line graph showing error spike at 2:30, and can click to see details

Think of it like: The difference between reading a spreadsheet of numbers vs. seeing colorful charts that tell a story!

Part 5: How They Work Together (The Complete Picture)

Think of it like a pizza delivery system:



-  Elasticsearch (Warehouse)
 - ↓ stores orders
-  Kibana (Manager Dashboard)
 - ↓ shows charts
-  You (Manager)
 - ↓ sees what's happening

Step-by-step flow:

1. **Your code:** `Log.Information("Order created for John")`
2. **Serilog:** Grabs this message, adds time, service name, environment
3. **Serilog:** Sends it to Elasticsearch (like sending to a super-fast database)
4. **Elasticsearch:** Stores it and makes it searchable
5. **Kibana:** Reads from Elasticsearch and shows beautiful charts

Real example from your project:

- Catalog API logs: "Product added: Adidas Shoes"
- Serilog adds: Time=2:30 PM, Service=Catalog.API, Environment=Development
- Elasticsearch stores it with index: `ecommerce-catalog-api-development-2025.04.29`
- Kibana shows you a chart: "10 products added today"

Part 6: Step-by-Step Implementation Guide

Step 1: Start the Infrastructure (The Foundation)

First, we need Elasticsearch and Kibana running. You already have them in your docker-compose!

```
# Run this command to start everything
docker-compose up -d elasticsearch kibana
```

What this does: Starts Elasticsearch (port 9200) and Kibana (port 5601) in Docker containers

Check if it's working:

- Open browser: <http://localhost:9200> (should see JSON from Elasticsearch)
- Open browser: <http://localhost:5601> (should see Kibana login screen)

Step 2: Add Serilog to Your Service (The Setup)

You already have the Common.Logging project! Now let's use it in one of your services.

Example: Enable Serilog in Ordering Service

Open Ordering.API/Program.cs and find line 57 where Serilog is commented out:

```
// BEFORE (commented out):  
//builder.Host.UseSerilog(Logging.ConfigureLogger);  
  
// AFTER (uncomment it):  
builder.Host.UseSerilog(Logging.ConfigureLogger);
```

That's it! Just one line enables Serilog.

Step 3: Add Configuration (Tell Serilog Where to Send Logs)

Open Ordering.API/appsettings.json and add this:

```
{  
  "ElasticConfiguration": {  
    "Uri": "http://localhost:9200"  
  }  
}
```

What this does: Tells Serilog "Send logs to Elasticsearch at this address"

Step 4: Use Serilog in Your Code (Write Logs!)

Now you can log anywhere in your code:

```
// In any controller or service  
using Serilog;
```

```
public class OrderController
```

```

{
    public IActionResult CreateOrder(Order order)
    {
        Log.Information("Creating order for user {UserName}",
order.UserName);

        try
        {
            // Your order logic here
            Log.Information("Order {OrderId} created successfully",
order.Id);
            return Ok();
        }
        catch (Exception ex)
        {
            Log.Error(ex, "Failed to create order for user
{UserName}", order.UserName);
            return BadRequest();
        }
    }
}

```

Step 5: Run and Test (See It Work!)

```

# Start your Ordering service
cd Services/Ordering/Ordering.API
dotnet run

```

What happens:

1. Your service starts
2. Serilog automatically connects to Elasticsearch
3. Every log message gets sent to Elasticsearch
4. You see logs in console AND in Elasticsearch

Step 6: View Logs in Kibana (The Cool Part!)

1. Open <http://localhost:5601> in browser

2. Click "Stack Management" → "Index Patterns"
3. Click "Create index pattern"
4. Type: ecommerce-* (this matches all your service logs)
5. Click "Next step" → select "@timestamp" → "Create index pattern"
6. Go to "Discover" to see your logs!

You'll see:

- Time of each log
- Service name (Catalog.API, Basket.API, etc.)
- Log level (Information, Warning, Error)
- Your actual log message
- Any extra data you logged

Step 7: Enable in All Services (Complete Setup)

Repeat Steps 2-3 for all your services:

Services to update:

- Catalog.API (already has Serilog enabled ✓)
- Basket.API (already has Serilog enabled ✓)
- Discount.API (needs Serilog added)
- Ordering.API (uncomment line 57)
- Payment.API (uncomment line 30)
- Identity.API (already has Serilog enabled ✓)

For services without Serilog, add to Program.cs:

```
using Common.Logging;  
using Serilog;
```

```
// Add this after var builder = WebApplication.CreateBuilder(args);  
builder.Host.UseSerilog(Logging.ConfigureLogger);
```

And add to appsettings.json:

```
{  
  "ElasticConfiguration": {
```

```
    "Uri": "http://elasticsearch:9200"
  }
}
```

Step 8: Create a Dashboard (See the Magic!)

In Kibana:

1. Go to "Visualize Library"
2. Click "Create visualization"
3. Choose "Line chart"
4. Select your index pattern (ecommerce-*)
5. Choose "Count" for Y-axis
6. Choose "@timestamp" for X-axis (by hour)
7. Click "Save" → name it "Error Rate"

Now you have a beautiful chart showing errors over time!

Quick Reference Card

Component	What it does	Where it runs
Serilog	Writes logs	Inside your .NET services
Elasticsearch	Stores and searches logs	Docker container (port 9200)
Kibana	Shows beautiful charts	Docker container (port 5601)

Common Log Levels

```
Log.Verbose("Super detailed info - only for debugging");
Log.Debug("Detailed info - development only");
Log.Information("Normal events - order created, user logged in");
Log.Warning("Something might be wrong - slow database");
Log.Error("Something went wrong - payment failed");
Log.Fatal("Critical - service is down!");
```


Your Current Setup (What You Have)

Looking at your Logging.cs file, you already have:

- ☒ Console logging (shows in terminal)
- ☒ Elasticsearch logging (sends to Elasticsearch)
- ☒ Automatic enrichment (adds service name, environment, time)
- ☒ Exception details (shows full error information)
- ☒ Per-service indexes (organized by service and date)

You just need to:

1. Uncomment Serilog in Ordering and Payment services
2. Add Serilog to Discount service
3. Add Elasticsearch URL to all appsettings.json files

Summary

You now understand the complete logging stack:

The Problem: When your microservices have issues, you need to know what happened

The Solution:

- **Serilog** = Smart notebook writer for your code
- **Elasticsearch** = Super-fast search engine for logs
- **Kibana** = Beautiful dashboard to visualize logs

Your Current Status: You already have the Common.Logging project set up perfectly! You just need to:

1. Uncomment Serilog in Ordering and Payment services
2. Add Serilog to Discount service
3. Ensure all services have the Elasticsearch URL in their config

Next Steps: Would you like me to help you enable Serilog in the remaining services (Discount, Ordering, Payment) by making the code changes?